**ВКТУ**

DEVOPS · 2-БАҚЫЛАУ НҮКТЕСІ · 3-АПТА

# 2-зертханалық жұмыс
# Dockerfile және құрастыру кэші

**1-бөлім: 1-5-тармақтар. 03-материал.**

**Орындаған:** Есімханова Аружан Түсіпханқызы
**Тобы:** 25-ИСТК-2о
**Күні:** 2026 жылғы 22 қазан

## Жұмыстың мақсаты

ASP.NET Core / .NET 10 негізіндегі DemoApi сервисін екі кезеңді құрастыру арқылы Linux бейнесіне жинақтау, артық файлдарды алып тастау және бастапқы код өзгергеннен кейін NuGet қабатының қайта пайдаланылуын растау.

| Орта | Нақты мәні |
|---|---|
| Docker Client / Server | 29.8.0 / 29.8.0 |
| Орындау ортасы | Docker Desktop, Linux x86_64, WSL 2 қосылған |
| Docker ресурстары | 12 CPU, 7.62 GiB жад (кемінде 4 GB талап етіледі) |
| PowerShell | 7.6.5 |
| Негізгі бейнелер | dotnet/sdk:10.0 және dotnet/aspnet:10.0 |

## Тапсыру материалдары

Dockerfile және .dockerignore толық берілген; екінші құрастыру шығысында restore қадамы үшін CACHED көрсетілген. hello-world іске қосылуы, Dockerfile, нақты контейнер және API тексерілді. Settings → General ішіндегі WSL 2 жалаушасының міндетті экран суреті әлі қажет: терезені автоматты түсіру уақыты асып кеткен. Конфигурация WSL 2 қосылғанын растайды, бірақ мәтін экран суретін алмастырмайды.

Ашық репозиторий және барлық бастапқы код

Ескерту: командалар, API жауаптары, журналдар және шынайы экран суреттеріндегі мәтін дәлел ретінде бастапқы тілінде сақталды. Түсіндірмелер мен қорытындылар қазақ тіліне аударылды.

# 1. Dockerfile: екі кезең

1-листинг. Репозиторий түбіріндегі толық Dockerfile. Алдымен тәуелділіктер, содан кейін бастапқы код көшіріледі; соңғы бейнеге publish нәтижесі қосылады.

```
# syntax=docker/dockerfile:1

FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY src/DemoApi/DemoApi.csproj src/DemoApi/
RUN dotnet restore src/DemoApi/DemoApi.csproj
COPY src/ src/
RUN dotnet publish src/DemoApi/DemoApi.csproj -c Release -o /app --no-restore

FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime
RUN apt-get update \
    && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY --from=build /app .
ARG APP_VERSION=dev
ENV APP_VERSION=$APP_VERSION
USER $APP_UID
EXPOSE 8080
HEALTHCHECK --interval=10s --timeout=3s --start-period=15s --retries=3 \
    CMD curl -fsS http://localhost:8080/health/live || exit 1
ENTRYPOINT ["dotnet", "DemoApi.dll"]
```

build кезеңі SDK пайдаланып, Release режимінде restore/publish орындайды. runtime кезеңі ASP.NET орындалу ортасын, жарияланған қолданбаны және жұмыс күйін тексеруге арналған curl құралын қамтиды. curl орнату мен apt тазалау бір қабатта біріктірілген.

USER $APP_UID сервисті app пайдаланушысы атынан іске қосады. EXPOSE ішкі 8080 портын құжаттайды. HEALTHCHECK /health/live маршрутын шақырады. ARG құрастырудың ашық нұсқасын береді, ENV оны GET / арқылы қолжетімді етеді. ENTRYPOINT dotnet-ті негізгі процесс ретінде іске қосады.

GitHub-тағы Dockerfile

## 2. Алып тасталатын файлдар және негізгі бейне

2-листинг. Толық .dockerignore. Нұсқаулықтағы үлгіге қосымша есептер, дәлелдер, құжаттама, Postman және ішкі каталогтардағы орта / сертификат файлдары алынып тасталған.

```
**/bin/
**/obj/
**/TestResults/
.git
.github
.vs
.vscode
.idea
tests/
monitoring/
scripts/
db/
docs/
reports/
evidence/
postman/
*.md
Dockerfile
compose*.yaml
compose*.yml
.env
.env.*
**/.env
**/.env.*
**/*.pfx
**/*.p12
```

bin/obj ішінде Linux құрастыруына кірмеуі тиіс жергілікті Windows артефактілері бар. Қолданбаны жариялау үшін Git, IDE, тестілер және есептер қажет емес. .env және сертификаттар уақытша қабаттарға да түспеуі үшін құрастыру контекстінен алынып тасталады.

Дайындық тексерулері: docker version Client және Server деректерін көрсетті; hello-world сәтті іске қосылды. Негізгі runtime бөлек жүктеліп, оның қабаттары мен өлшемдері image history және image ls командаларымен тіркелді. Docker build --check ескертусіз аяқталды.

Дайындық пен тексерудің толық журналдары

# 3. Құрастыру және кэшті қайта пайдалану

Алдымен 1.0 нұсқасы құрастырылды. Кейін TaskService.cs ішінде тек тапсырма жасау хабарламасы өзгертілді; жоба файлы мен пакеттер тізімі өзгерген жоқ. Екінші құрастыруға 1.1 тегі және APP_VERSION=lab02-1.1 мәні берілді.

```
docker build --progress=plain -t devops-labs-demo-api:lab02-1.0 .
docker build --progress=plain --build-arg APP_VERSION=lab02-1.1 `
    -t devops-labs-demo-api:lab02-1.1 .
```

### Хабарламаның өзгерісі (соңғы кодта сақталған):

```
Создана задача {TaskId} в проекте {ProjectId}
Создана новая задача {TaskId} в проекте {ProjectId}
```

### 3-листинг. Екінші docker build командасының нақты үзіндісі:

```
#12 [build 3/6] COPY src/DemoApi/DemoApi.csproj src/DemoApi/
#12 CACHED
#13 [build 4/6] RUN dotnet restore src/DemoApi/DemoApi.csproj
#13 CACHED
#14 [build 5/6] COPY src/ src/
#14 DONE 0.0s
#15 [build 6/6] RUN dotnet publish src/DemoApi/DemoApi.csproj -c Release -o /app --no-restore
#15 3.234   DemoApi -> /src/src/DemoApi/bin/Release/net10.0/DemoApi.dll
#15 3.289   DemoApi -> /app/
#15 DONE 3.4s
```

Жоба файлы мен restore нәтижесі кэштен алынды; бастапқы кодты көшіру және publish қайта орындалды. Демек, кодтың өзгеруі NuGet пакеттерін қайта жүктеуді қажет етпеді. Өзгеріске дейінгі және кейінгі SHA-256, сондай-ақ қадамдардың тексеру нәтижелері cache-check.json файлында сақталған.

| Артефакт | Нақты мәні |
|---|---|
| Соңғы бейне | devops-labs-demo-api:lab02-1.1 |
| Бейне өлшемі (image inspect) | 354,253,942 байт (337.8 MiB) |
| Платформа / пайдаланушы | linux/amd64, UID 1654 |

# 4. Құрастырылған қолданбаны тексеру

Бұл тексерулер міндетті 1-5-тармақтарды толықтырады. API нақты Linux контейнерінде localhost:8089 → 8080 арқылы іске қосылды. Тексеру үшін басқа сервистер мен контейнерлер пайдаланылмады.

| Тексеру | Нәтиже |
|---|---|
| Контейнердің жұмыс күйі | healthy; /health/live контейнер ішінен орындалады |
| Қолданба нұсқасы | GET / → 200, version=lab02-1.1 |
| Процесс пайдаланушысы | uid=1654(app) gid=1654(app) groups=1654(app) |
| Соңғы бейнедегі SDK | dotnet --list-sdks: бос шығыс |
| Бейнедегі бастапқы код және тестілер | /app ішінде C# бастапқы кодтары, csproj және шешім файлы жоқ |
| Live және ready | Екі маршрут та → 200 |
| Production ортасындағы Swagger | GET /swagger/index.html → 404 |
| InMemory қайта іске қосылуы | Сол контейнерде: stop/start алдында 1 жоба, кейін 0 |
| Қайта құрастырмай Swagger қосу | Сол бейне Development ортасында: /swagger/index.html → 200 |
| Development ортасындағы OpenAPI | GET /openapi/v1.json → 200, stats бар |
| Оқу ақаулары өшірілген | Chaos__Enabled=false, /api/chaos/error → 404 |
| API HTTP сценарийлері | 26 сұрау: 23 сценарий және тестілік жобаларды жоюға арналған 3 сұрау |
| Postman CLI коллекциясы | 24 сұрау, 73 тексеру, 0 қате; аяқталу коды 0 |
| .NET Release регрессиялық тексеруі | 28/28 тест, 0 қате, 0 өткізіп жіберілген; InMemory |

### GET / жауабы:

```
{
  "service": "DemoApi",
  "version": "lab02-1.1",
  "host": "e7ee83006c4d"
}
```

Толық сұраулар мен жауаптар, контейнер журналдары, жұмыс күйі және Postman нәтижелері evidence/lab02 каталогында сақталған. API жергілікті тексерілді; нәтижелер Postman бұлтына жарияланбаған.

# 5. Сұрақтарға жауап және қорытынды

### Контейнер неге виртуалды машинадан жылдам іске қосылады?

Бөлек ядро мен толық операциялық жүйені жүктеу қажет емес. WSL 2 ішінде Linux ядросы жұмыс істеп тұр; Docker namespaces/cgroups жасайды, қабаттарды тіркейді және қолданба процесін іске қосады.

### Неге алдымен csproj, содан кейін код көшіріледі?

Тәуелділіктер тізімі C# файлдарына қарағанда сирек өзгереді. Код өзгергеннен кейін restore-дың бөлек қабаты кэште қалады. Бұл екінші құрастырумен расталған.

### Көп кезеңді құрастырудан кейін соңғы бейнеге не кіреді?

ASP.NET орындалу ортасы, жарияланған DLL файлдары мен тәуелділіктер, конфигурация және curl. SDK, бастапқы кодтар мен аралық obj файлдары build кезеңінде қалады. SDK жоқтығы dotnet --list-sdks арқылы тексерілген.

### Контейнерде Swagger неге жоқ? Оны қалай қосуға болады?

Әдепкі орта - Production; Program.cs Swagger-ді тек Development ортасында тіркейді. Сол бейнеден ASPNETCORE_ENVIRONMENT=Development мәнімен жасалған жаңа контейнер Swagger-ді қайта құрастырусыз ашты. Chaos__Enabled=false оқу ақауларын өшіреді.

### docker stop/start орындалғаннан кейін жобаларға не болады?

InMemory процесс жедел жадында сақталады. Процесс аяқталғанда контейнер сол күйі қалса да, жобалар жоғалады. Нақты тексеруде жоба жасалып, контейнер тоқтатылып / іске қосылды, содан кейін бос тізім алынды.

### Қорытынды және шектеу

03-материалдың техникалық бөлігі орындалды және сақталған іске қосу нәтижелерімен расталды. Бақылау нүктесін толық рәсімдеу үшін Docker Desktop Settings → General бөліміндегі WSL 2 қосылғанын көрсететін шынайы экран суреті қажет. Бөлек 2-бөлім (04-материал) бастапқы мұрағатта жоқ және орындалды деп көрсетілмейді.

### Дереккөздер және материалдар

Шарттар: жүктелген Devops (1).zip мұрағатындағы 03-материал, «Неделя 3 - Контрольная точка 2», 2-зертханалық жұмыстың 1-бөлімі және 2-дәріс.

Docker: кэш ережелері · Microsoft: ASP.NET Core бейнелері

Репозиторийдегі нұсқаулар мен жауаптар · Сақталған дәлелдер